Benchmarking Quantization for Real-Time HEP Trigger Classifiers

Piush Kole

2026-10-15

Abstract

Real-time Level-1 trigger systems at the LHC must classify jets within microseconds on FPGA hardware with hard resource ceilings, making neural-network quantization a practical necessity rather than an optimization. We present a benchmarking study of post-training quantization (PTQ) and quantization-aware training (QAT) for a jet-tagging classifier, extending a uniform-precision baseline with (1) a near-exhaustive per-unit mixed-precision search that finds a layout reliably beating uniform INT6 PTQ at lower storage, (2) an independent hardware-resource estimate trained on real external synthesis data rather than a hand-derived heuristic alone, (3) a systematic comparison against non-neural-network baselines, and (4) a cross-architecture, cross-dataset generality check using a convolutional network on pixelated calorimeter images instead of hand-engineered tabular features. We report what each result does and does not support, including two findings that run against the simplest narrative: a gradient-boosted-tree baseline matches or exceeds both neural architectures tried, and post-training quantization — reliable for the tabular MLP — catastrophically fails for the CNN even at INT8, while quantization-aware training recovers gracefully in both cases.

1. Introduction and Motivation

At the Large Hadron Collider, the Level-1 trigger must decide, within microseconds, which of ~40 million collisions per second are worth recording (Duarte et al. 2018). Machine-learned classifiers increasingly replace or augment hand-written trigger logic, but any such model must fit within an FPGA’s fixed timing

and resource budget — making quantization (reducing numeric precision from FP32 to INT8/INT6/INT4) a deployment necessity, not a convenience.

The central empirical question this work asks is simple to state and not simple to answer cleanly: **how much precision can be removed before physics performance becomes unacceptable, and does the answer depend on the quantization strategy, the search granularity, the measurement method used to validate the hardware cost, or the model architecture itself?** Each of these axes turns out to matter, and the honest answer to "does X help" is more often "it depends, and here is exactly how" than a clean yes or no.

2. Related Work

This project sits in a well-established HEP-ML lineage. The jet-tagging benchmark used throughout (Section 3) is the hls4ml LHC jet dataset of Duarte et al. (Duarte et al. 2018), the same group behind `hls4ml` itself (hls4ml collaboration 2024), the FPGA-deployment toolchain this work targets. The particle-level image companion dataset used for the generality check (Section 7) supplements Moreno et al.'s JEDI-net work (Moreno et al. 2020). Automatic mixed-precision quantization for this exact niche has advanced substantially in the broader field: QKeras/AutoQKeras (Coelho et al. 2021) pioneered gradient-free per-layer mixed precision validated against real synthesis, and HGQ/HGQ2 (Sun et al. 2024) is the current state of the art, using a gradient-based, per-parameter bit-width search via a differentiable resource proxy. More recent architecture-side work has pushed hls4ml-deployable models beyond MLPs toward GNNs and transformers for jet tagging (Sun et al. 2025), and CMS's WOMBAT system (Moreno et al. 2025) demonstrates a Master/Apprentice (full-precision teacher, quantized FPGA-deployable student) design for a real Phase-2-trigger use case. Most relevant to the hardware-validation thread of this work, wa-hls4ml (Hawks et al. 2025) is a very recent (November 2025) public dataset of ~680,000 real hls4ml-synthesized networks with pretrained surrogate models — this project trains its own, smaller surrogate on the same public data rather than adopting the authors' own model directly, for reasons discussed in Section 6.

This project's own contribution is not a new quantization method — it uses standard uniform affine fake-quantization (PTQ) and straight-through-estimator training (QAT), the same mechanisms underlying most of the literature above. The contribution is **benchmarking methodology and rigor**: a near-exhaustive (not hand-picked) mixed-precision search at full statistical power, an independently-trained hardware-cost check rather than a single

heuristic, systematic non-neural baselines, and an explicit test of whether conclusions drawn from one architecture and dataset transfer to another.

3. Data

Primary (tabular) dataset: the hls4ml LHC jet-tagging benchmark (Duarte et al. 2018) (OpenML `hls4ml_lhc_jets_hlf` v1, DOI 10.5281/zenodo.3602260) — 16 hand-engineered high-level substructure features (energy-correlation functions, N-subjettiness ratios, jet mass variants) per jet, 5-class label (gluon/quark/top/W/Z), near-balanced (19.4-20.2% per class, confirmed by direct count, not assumed). 100,000-jet stratified subsample, 70/15/15 train/val/test split, 5 seeds `{42, 123, 2026, 7, 99}`.

Generality-check (image) dataset: a particle-level companion to the same benchmark (Zenodo 10.5281/zenodo.3601436, CC BY 4.0, supplement to (Duarte et al. 2018) and (Moreno et al. 2020)) — the same underlying Pythia8 simulation and 5-class label, but with pre-rendered 100×100 calorimeter-energy images per jet instead of hand-engineered features. Confirmed by direct inspection of the downloaded HDF5 schema (not assumed): a single combined calorimeter-energy channel is used (separate ECAL/HCAL channels exist, but ECAL is ~4e-6 nonzero for these hadronic jets, confirmed directly — a near-empty second channel would add nothing). Official `train`/`val` split used as shipped (not re-split): `train` subsampled to 100,000 jets and further split 85/15 internally; the full 270,000-jet `val` split held out entirely as test, touched once.

4. Method

4.1 Quantization mechanism

Both PTQ and QAT use uniform affine fake-quantization (quantize-then-dequantize, computed in float32) rather than real bit-packed integer kernels, since PyTorch’s native quantized kernels only support INT8, not INT6/INT4 — this is a standard, technically valid research methodology (the same mechanism underlies most QAT literature) but does not produce a packed artifact; model size is computed analytically. QAT uses a straight-through estimator so gradients flow through the non-differentiable rounding operation, with activation ranges tracked via an exponential moving average observer during training — the standard QAT recipe.

4.2 Architectures

MLP (primary): `16 → [hidden] → 5`, three configurations tested across the project (a `[128,64,32]` base architecture plus two further shapes for an architecture-generalization check, 4,389–16,837 parameters). **CNN** (generality check, Section 7): 3 convolutional blocks (16→32→64 channels, 3×3 kernels, 2×2 max-pool) over 25×25 downsampled images, followed by a small fully-connected classifier head, ~90,000 parameters.

4.3 Experimental discipline

Every result in this work follows the same discipline: a selection rule (which configuration "wins") is written to disk *before* the test split is touched, non-NN baselines are trained before any neural result is presented as the headline, every reported number is tagged with a measurement tier (measured / simulated / estimated / surrogate / unavailable) so a reader never has to guess how trustworthy it is, and every deterministic result is verified by running the same code twice and diffing the output byte-for-byte (the project's "two-test rule").

5. Core Result 1: A Near-Exhaustive Mixed-Precision Search Changes the Conclusion

An earlier, smaller search over only 9 hand-picked per-unit bit-width layouts (varying each of the model's 7 quantizable units — 4 weight tensors, 3 activation tensors — individually and in two combinations) concluded there was "no advantage over uniform INT6 PTQ." That search could not have found the result below, because its candidate set did not include it.

A true near-exhaustive search — all 3^7 = 2,187 per-unit bit-width combinations ({INT4, INT6, INT8} per unit), evaluated at full 5-seed statistical power (confirmed computationally cheap enough — 0.3 estimated hours — to run directly rather than needing a staged screening shortcut) — finds a genuine Pareto frontier of 23 non-dominated layouts, not a single point. The frontier's QAT-fine-tuned selection (storage **2,916 bytes**, picked by a pre-registered rule: highest mean validation ROC-AUC among layouts at or under uniform INT6's storage budget) **reliably beats uniform INT6 PTQ** (storage 3,724 bytes) by +0.0013 mean ROC-AUC, 95% CI [0.0011, 0.0015] entirely positive across all 5 seeds, while using **22% less storage** — and is statistically indistinguishable from the FP32 baseline. This revises, rather than confirms, the earlier 9-candidate search's conclusion.

configuration	storage (B)	mean test ROC-AUC
FP32	17,556	0.9343
INT8 PTQ	4,788	0.9342
INT6 PTQ	3,724	0.9329
INT4 PTQ	2,660	0.8813
INT4 QAT	2,660	0.9246
Mixed PTQ (this search)	2,996	0.9321
Mixed QAT (this search)	**2,916**	**0.9342**

A genuine engineering reproducibility gap was found and fixed in the course of this work: the per-unit QAT fine-tuning builder did not pin a random seed, causing a small (~3×10^{-4} ROC-AUC) but real run-to-run drift inconsistent with the project's reproducibility discipline. Fixed by seeding before fine-tuning, re-verified to byte-identical reruns.

6. Core Result 2: An Independent Hardware Estimate, and Its Own Found Limitation

The project's only prior hardware-resource estimate was a hand-derived analytical heuristic (parameter count × bit-width × reuse factor), explicitly never validated against real synthesis (no Vivado/Vitis HLS license available in this environment). To ground at least one estimate in real synthesis data without requiring local Vivado access, a small sklearn surrogate regressor was trained on `fastmachinelearning/wa-hls4ml` (Hawks et al. 2025), a public dataset of ~680,000 real hls4ml-synthesized networks (training restricted to its `resource/latency` fully-connected splits, excluding splits the dataset's own card flags as unrepresentative).

The surrogate generalizes well for latency/II (R^2 up to 0.996 on held-out data) but — a limitation found and disclosed, not hidden — **gives essentially zero feature importance to bit-width or layer width**, for every target, under both a random-forest and a gradient-boosted model (verified, not assumed, by inspecting feature importances directly and by confirming two feature vectors differing only in precision get identical predictions). Two models of the same architecture at different precisions receive the same surrogate LUT/DSP/FF/BRAM estimate. The likely cause: `total_params` and `reuse_factor` alone explain most resource variance across the training data's extremely heterogeneous architecture mix, leaving too little residual signal for a tree ensemble to bother splitting on bit-width. **The original heuristic — cruder, but explicitly bit-width-sensitive — remains the**

more useful of the two for precision comparisons; the surrogate's validated latency/II numbers are its genuine contribution.

7. Core Result 3: Does the Neural Network Earn Its Keep?

A question nothing in this project asked until explicitly tested: is a neural network actually the best available model for this task? Two default-hyperparameter (no tuning) baselines were trained on the identical splits as the MLP: logistic regression and `HistGradientBoostingClassifier` (GBM).

The GBM baseline reliably beats the FP32 MLP on ROC-AUC (+0.0030, 95% CI [0.0018, 0.0041] entirely positive, all 5 seeds) and on background rejection at 70% signal efficiency. Logistic regression, as expected, is clearly worse than either nonlinear model. **This finding is replicated independently on the image/CNN generality check** (Section 8): a GBM trained on hand-engineered image features (total energy, centroid, spread, nonzero-pixel count) reliably beats the CNN (-0.0044, 95% CI entirely negative, 0/3 seeds, n=3) while the CNN in turn reliably beats a flattened-pixel logistic-regression control (+0.1255, 3/3 seeds). Across two independent architectures and two independent data representations, a simple gradient-boosted-tree baseline on hand-engineered features matches or exceeds the neural network. **Neither result claims the relevant neural network is the best available model for its task** — the case for the quantized MLP specifically is FPGA deployability via `hls4ml`, a path a `HistGradientBoostingClassifier` is not built to use; the honest conclusion is that the case for the neural network is deployability on the target hardware path, not raw accuracy.

8. Generality Check: A CNN on a Different Architecture and Dataset

A second architecture family (CNN, not MLP) on a second data representation (pixelated calorimeter images, not hand-engineered features) was built to test whether this project's methodology — and the quantization behavior observed for the MLP — transfer. This is reported as a secondary generality check, not a second headline result: the quantization/mixed-precision work above remains this project's core, tightly-scoped contribution.

Baseline comparison (Section 7) showed the CNN sits between the two baselines, as described above. **Quantization behavior diverges sharply from the MLP's**, and this divergence is itself the generality check's most important finding:

config	ROC-AUC	accuracy	note
FP32	0.8836	0.6246	—
INT8 PTQ	0.7323	**0.2016**	collapsed (5-class random ≈ 0.20)
INT8 QAT	0.8812	0.6272	within noise of FP32
INT6 PTQ	0.7161	**0.2015**	collapsed
INT6 QAT	0.8597	0.5904	graceful degradation
INT4 PTQ	0.6270	**0.2015**	collapsed
INT4 QAT	0.8234	0.5282	graceful degradation

Post-training quantization catastrophically fails for the CNN at every precision tried, including INT8 — accuracy collapses to ~0.20 (random-chance level for 5 near-balanced classes) and macro-F1 to ~0.067, a pattern closely matching a degenerate "always predicts one class" collapse. Notably, **ROC-AUC stays well above random (0.63-0.73)** even as the argmax decision collapses, meaning the underlying probability scores retain real ranking signal — consistent with PTQ distorting the decision *scale* (e.g. one class's logit dominating after quantization) while leaving relative ordering partially intact. **Quantization-aware training recovers gracefully** in every case, the way QAT is designed to.

On the tabular MLP, by contrast, INT8 PTQ was *practically lossless* (Section 1 table) and only INT4 PTQ showed real instability. **This is a genuine, substantive generality finding: PTQ reliability is not architecture-independent.** A conclusion drawn from the MLP work alone — "INT8 PTQ is usually safe for this kind of model" — does not transfer to the CNN on this task. A candidate explanation (Conv2D weight-range outliers dominating the naive min/max calibration, or an insufficient activation-calibration sample size for a spatially-extended feature map) is stated as a hypothesis in the underlying phase report, not confirmed by further investigation — disclosed as exactly that, an open question this work surfaces rather than resolves.

This generality check also required (and delivered, as a secondary infrastructure contribution) extending the project's PTQ/QAT machinery, previously hardcoded to the MLP's flat single-`Sequential` structure, to a model with two separate `Sequential` blocks (convolutional and classifier) and `Conv2D` layers — a generalization verified not to change the existing MLP code path's behavior (regression-tested), and verified to work end-to-end against the real (not mocked) `hls4ml` PyTorch-to-HLS conversion path for `Conv2D`/`MaxPooling2D` layers.

9. Discussion and Limitations

What this work supports: that a near-exhaustive mixed-precision search materially changes conclusions a smaller hand-picked search reaches; that an externally-trained hardware estimate can be a genuine, if imperfect, complement to a hand-derived heuristic, with its own limitations disclosed rather than hidden; that the case for a quantized neural network on this task is deployability, not accuracy, on both datasets tried; and that quantization reliability is not guaranteed to transfer across architectures even on the same underlying physics task.

What this work does not support: any claim of a novel quantization *method* (standard PTQ/ QAT mechanisms throughout); any physical FPGA latency or resource claim (no Vivado/Vitis HLS synthesis was performed — every hardware number is simulated, estimated, or a trained surrogate, never measured on real hardware); a confirmed mechanism for the CNN's PTQ collapse (stated as an open hypothesis); or statistical significance in the formal sense for comparisons at n=3 seeds (the CNN quantization sweep itself, Section 8, used a single seed, disclosed as such given the CNN's measured per-seed training cost).

Honesty discipline throughout: every number in this work is tagged with how it was obtained (measured in software, simulated via hls4ml C-simulation, an analytical estimate, a trained surrogate with its own held-out error, or genuinely unavailable), and every comparison's selection rule was fixed before the test split was examined. Where a result ran against the simplest or most flattering narrative — a baseline beating the neural network, a found reproducibility bug, a surrogate model's blind spot, quantization collapsing on the second architecture — it is reported as found, not reframed or omitted.

10. Conclusion

This work set out to benchmark quantization for a real-time HEP trigger classifier with the rigor the deployment stakes warrant: a true search instead of a hand-picked one, an independent hardware check instead of a single heuristic, honest baselines instead of an assumed neural- network advantage, and an explicit test of whether any of this generalizes. Each of these axes surfaced a result worth reporting precisely because it was not the simplest possible outcome: mixed-precision QAT beats uniform INT6 at lower storage; the externally-trained hardware surrogate cannot currently see precision at all; a gradient-boosted-tree baseline matches or beats both neural architectures tried; and PTQ's reliability, solid for the MLP, collapses for the CNN. Together, these support a central, generalizable practical claim for anyone quantizing a model for FPGA trigger deployment: **validate the specific method, the specific search granularity, and the specific architecture together** — conclusions from any one of these axes in isolation do not reliably transfer to the others.

Reproducibility

Every result in this paper traces to a specific phase report and reproduction command in the accompanying repository (`hep-trigger-quantization-lab`): Section 5 to `reports/phase12_exhaustive_mixed_precision_pareto.md`, Section 6 to `reports/phase11_surrogate_hardware_validation.md`, Section 7 to `reports/phase13_baselines_and_efficiency.md` and `reports/phase14_cnn_jet_images.md`, and Section 8 to `reports/phase14_cnn_jet_images.md` and `reports/phase14_cnn_quantization.md`. Pinned dependencies, exact reproduction commands, and the full two-test-rule verification record for every result are in `docs/REPRODUCIBILITY.md`.

References

Coelho, Claudionor N. et al. 2021. "Automatic Heterogeneous Quantization of Deep Neural Networks for Low-Latency Inference on the Edge for Particle Detectors." *Nature Machine Intelligence* 3: 675–86. https://arxiv.org/abs/2006.10159.

Duarte, Javier et al. 2018. "Fast Inference of Deep Neural Networks in FPGAs for Particle Physics." *Journal of Instrumentation* 13: P07027. https://arxiv.org/abs/1804.06913.

Hawks, Ben et al. 2025. "Wa-Hls4ml: A Public Dataset of Real Hls4ml Synthesis Results for Surrogate Resource Modeling." https://arxiv.org/abs/2511.05615.

hls4ml collaboration. 2024. “Hls4ml: FPGA Deployment Toolchain for Machine Learning in High-Energy Physics.” https://fastmachinelearning.org/hls4ml/.
Moreno, Eric A. et al. 2020. “JEDI-Net: A Jet Identification Algorithm Based on Interaction Networks.” *European Physical Journal C* 80: 58. https://arxiv.org/abs/1908.05318.
——— et al. 2025. “WOMBAT: A Trigger-Level Jet-Substructure Tagging System.” https://arxiv.org/abs/2505.05532.
Sun, Chang et al. 2024. “HGQ2: Gradient-Based Mixed-Precision Quantization for Hardware-Efficient Neural Networks.” https://arxiv.org/abs/2405.00645.
——— et al. 2025. “JEDI-Linear: Jet Identification via Linear Interaction Networks.” https://arxiv.org/abs/2508.15468.